

C++ Memory Management and Object Semantics

1. Overview

C++ gives programmers direct control over memory and object lifetime. This is powerful, but it also means mistakes can cause crashes, memory leaks, dangling pointers, or undefined behavior.

This topic covers:

- Pointer vs reference
- Stack vs heap
- `malloc` vs `new`
- `delete` vs `free`
- RAI
- Smart pointers
- `unique_ptr` vs `shared_ptr`
- Memory leaks and dangling pointers
- `vector` resizing
- `map` vs `unordered_map`
- Virtual functions and virtual destructors
- Copy vs move

Definition

C++ memory management is the process of creating, using, owning, copying, moving, and destroying objects safely and efficiently.

Why it matters

In C++, memory bugs are common interview topics because they test whether you understand:

- Object lifetime
- Ownership
- Performance
- Resource cleanup
- Runtime behavior
- Undefined behavior

Where it is used in real systems

These concepts are used in:

- Operating systems
- Game engines
- Browsers
- Backend servers
- Databases
- Embedded systems

- High-frequency trading systems
- Compilers

Why interviewers ask about it

Interviewers ask these topics because they reveal whether you can write safe and efficient C++ code. Many bugs in production C++ systems happen due to wrong ownership, incorrect deletion, invalid references, or expensive copying.

2. Core Idea

The core idea is simple:

In C++, every object has a lifetime, a storage location, and an owner.

If you understand where an object lives, who owns it, and when it is destroyed, most C++ memory questions become easy.

Intuition

Think of memory like rooms in a building:

- Stack memory is like a temporary meeting room. You enter, do your work, and leave automatically.
- Heap memory is like renting a storage unit. You must explicitly release it, or use an automatic manager.
- RAII is like hiring a responsible manager who opens and closes resources for you.
- Smart pointers are like ownership documents for heap memory.

Small Example

```
#include <iostream>
#include <memory>

void example() {
    int x = 10; // stack
    int* p = new int(20); // heap, manual ownership
    std::unique_ptr<int> q = std::make_unique<int>(30); // heap, automatic
    ownership

    std::cout << x << " " << *p << " " << *q << "\n";

    delete p; // manual cleanup
} // x and q are cleaned automatically
```

Step-by-step explanation

1. `x` is created on the stack.
2. `p` points to memory created on the heap using `new`.
3. `q` owns heap memory using `unique_ptr`.
4. `delete p` manually releases `p`.

5. When the function ends, `x` is destroyed automatically.
6. `q` also automatically deletes its owned heap object.

Interviewers mainly want to know whether you understand the difference between automatic cleanup and manual cleanup.

3. Important Subtopics

3.1 Pointer vs Reference

What it means

A pointer stores the address of another object. A reference is another name for an existing object.

```
int x = 10;
int* p = &x;    // pointer to x
int& r = x;     // reference to x
```

Why it matters

Pointers and references are used for passing objects efficiently, modifying data, implementing data structures, and enabling polymorphism.

Example

```
void incrementByPointer(int* p) {
    if (p != nullptr) {
        (*p)++;
    }
}

void incrementByReference(int& r) {
    r++;
}
```

Common interview angle

Interviewers ask:

- Can a pointer be null?
- Can a reference be null?
- Can a pointer be reassigned?
- Can a reference be reseated?

Key answer:

- Pointer can be null and reassigned.

- Reference should always refer to a valid object and cannot be resealed after initialization.

3.2 Stack vs Heap

What it means

Stack memory is automatically managed and usually stores local variables. Heap memory is dynamically allocated and must be managed by the programmer or by smart pointers.

```
void f() {  
    int a = 10;           // stack  
    int* b = new int(20); // heap  
    delete b;  
}
```

Why it matters

Choosing stack or heap affects performance, lifetime, and safety.

Example

```
std::string createName() {  
    std::string name = "Asha"; // stack object manages internal heap memory  
    return name;  
}
```

Common interview angle

Interviewers often ask whether stack is always faster than heap. Usually, stack allocation is faster because it only adjusts the stack pointer. Heap allocation needs allocator bookkeeping and may be slower.

3.3 malloc vs new

What it means

`malloc` is a C-style memory allocation function. `new` is a C++ operator that allocates memory and calls the constructor.

```
int* p1 = (int*)malloc(sizeof(int));  
int* p2 = new int(10);
```

Why it matters

In C++, objects need constructors and destructors. `malloc` does not call constructors. `new` does.

Example

```
class Student {
public:
    Student() {
        std::cout << "Constructor called\n";
    }
};

Student* s1 = new Student(); // constructor called
Student* s2 = (Student*)malloc(sizeof(Student)); // constructor not called
```

Common interview angle

Expected answer:

- `new` allocates memory and constructs the object.
- `malloc` only allocates raw bytes.
- `new` returns a typed pointer.
- `malloc` returns `void*`.
- `new` throws `std::bad_alloc` on failure.
- `malloc` returns `NULL` on failure.

3.4 delete vs free

What it means

`delete` releases memory allocated by `new` and calls the destructor. `free` releases memory allocated by `malloc` and does not call the destructor.

```
int* p = new int(5);
delete p;

int* q = (int*)malloc(sizeof(int));
free(q);
```

Why it matters

Mixing them causes undefined behavior.

Common interview angle

Never do this:

```
int* p = new int(10);
free(p); // wrong
```

```
int* q = (int*)malloc(sizeof(int));
delete q; // wrong
```

Use:

- `new` with `delete`
- `new[]` with `delete[]`
- `malloc` with `free`

3.5 RAII

What it means

RAII means Resource Acquisition Is Initialization.

It means a resource is acquired in a constructor and released in a destructor.

Why it matters

RAII prevents leaks even when exceptions happen.

Example

```
class FileHandle {
public:
    FileHandle() {
        std::cout << "Open file\n";
    }

    ~FileHandle() {
        std::cout << "Close file\n";
    }
};

void process() {
    FileHandle file;
    // file is automatically closed when function ends
}
```

Common interview angle

Interviewers expect you to connect RAII with:

- Destructors
- Exception safety
- Smart pointers
- Locks

- File handles
- Database connections

3.6 Smart Pointers

What it means

Smart pointers are C++ objects that manage heap memory automatically.

Common smart pointers:

- `std::unique_ptr`
- `std::shared_ptr`
- `std::weak_ptr`

Why it matters

Smart pointers reduce manual `new` and `delete`, making code safer.

Example

```
std::unique_ptr<int> p = std::make_unique<int>(10);
```

No `delete` is required.

Common interview angle

Interviewers ask which smart pointer to use:

- Use `unique_ptr` for single ownership.
- Use `shared_ptr` for shared ownership.
- Use `weak_ptr` to observe a `shared_ptr` object without increasing ownership count.

3.7 `unique_ptr` vs `shared_ptr`

What it means

`unique_ptr` means exactly one owner. `shared_ptr` means multiple owners.

```
std::unique_ptr<int> a = std::make_unique<int>(10);  
  
std::shared_ptr<int> b = std::make_shared<int>(20);  
std::shared_ptr<int> c = b;
```

Why it matters

Ownership affects performance, design, and object lifetime.

Example

```
std::unique_ptr<int> p1 = std::make_unique<int>(5);  
// std::unique_ptr<int> p2 = p1; // not allowed  
std::unique_ptr<int> p2 = std::move(p1); // ownership transferred
```

Common interview angle

Expected answer:

- `unique_ptr` is lightweight and move-only.
- `shared_ptr` uses reference counting.
- `shared_ptr` has overhead.
- Circular references with `shared_ptr` can cause memory leaks.
- Use `weak_ptr` to break cycles.

3.8 Memory Leak

What it means

A memory leak happens when heap memory is allocated but never released.

```
void leak() {  
    int* p = new int(10);  
} // memory leaked
```

Why it matters

Leaks reduce available memory and can crash long-running systems.

Example

Backend servers, browsers, and databases must run for a long time. Even a small leak can become serious over days or weeks.

Common interview angle

Interviewers ask how to prevent leaks:

- Use RAI.
- Use smart pointers.
- Match allocation and deallocation.
- Avoid raw owning pointers.
- Use tools like Valgrind, AddressSanitizer, or LeakSanitizer.

3.9 Dangling Pointer

What it means

A dangling pointer points to memory that is no longer valid.

```
int* p = new int(10);
delete p;
std::cout << *p; // dangling pointer usage, undefined behavior
```

Why it matters

Dangling pointers cause crashes, data corruption, and security bugs.

Example

```
int* getPointer() {
    int x = 10;
    return &x; // wrong: x dies when function returns
}
```

Common interview angle

Expected answer:

- Do not return address of local variables.
- Set raw pointer to `nullptr` after deleting if it may be reused.
- Prefer smart pointers and references with clear lifetime.

3.10 `vector` Resizing

What it means

`std::vector` stores elements in contiguous memory. When its capacity is full and a new element is inserted, it allocates a larger block, moves or copies elements, and frees the old block.

```
std::vector<int> v;
v.push_back(1);
v.push_back(2);
```

Why it matters

Vector resizing affects performance and invalidates pointers, references, and iterators.

Example

```
std::vector<int> v;  
v.push_back(10);  
  
int* p = &v[0];  
v.push_back(20); // may reallocate  
  
// p may now be dangling
```

Common interview angle

Interviewers ask:

- What happens when vector capacity is exceeded?
- Are iterators invalidated?
- What is the amortized complexity of `push_back`?

Expected answer:

- Reallocation may happen.
- Existing elements are moved or copied.
- Old pointers, references, and iterators may become invalid.
- `push_back` is amortized $O(1)$.

3.11 `map` vs `unordered_map`

What it means

`std::map` is usually implemented as a balanced binary search tree. `std::unordered_map` is implemented as a hash table.

Why it matters

Choice affects ordering, lookup complexity, memory usage, and worst-case performance.

Example

```
std::map<int, std::string> ordered;  
std::unordered_map<int, std::string> hashed;
```

Common interview angle

Expected answer:

- `map` keeps keys sorted.
- `unordered_map` does not keep keys sorted.
- `map` operations are $O(\log n)$.

- `unordered_map` average operations are $O(1)$, worst case $O(n)$.

3.12 Virtual Function

What it means

A virtual function allows runtime polymorphism. The function called depends on the actual object type, not just the pointer or reference type.

```
class Animal {
public:
    virtual void speak() {
        std::cout << "Animal\n";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        std::cout << "Dog\n";
    }
};

Animal* a = new Dog();
a->speak(); // Dog
delete a;
```

Why it matters

Virtual functions are used in interfaces, plugins, frameworks, and object-oriented design.

Common interview angle

Interviewers expect:

- Virtual functions enable dynamic dispatch.
- They usually use a vtable internally.
- They have slight runtime overhead.
- Use `override` to avoid mistakes.

3.13 Virtual Destructor

What it means

A base class destructor should be virtual if objects may be deleted through a base class pointer.

```
class Base {
public:
    virtual ~Base() = default;
```

```
};

class Derived : public Base {
public:
    ~Derived() {
        std::cout << "Derived destroyed\n";
    }
};

Base* p = new Derived();
delete p; // correct because Base destructor is virtual
```

Why it matters

Without a virtual destructor, deleting a derived object through a base pointer can cause undefined behavior.

Common interview angle

Expected answer:

If a class has virtual functions and is meant to be used polymorphically, it should almost always have a virtual destructor.

3.14 Copy vs Move

What it means

Copying duplicates a resource. Moving transfers ownership of a resource.

```
std::string a = "hello";
std::string b = a;           // copy
std::string c = std::move(a); // move
```

Why it matters

Move semantics improve performance by avoiding expensive deep copies.

Example

```
std::vector<int> makeVector() {
    std::vector<int> v = {1, 2, 3};
    return v;
}
```

Modern C++ can move or elide copies when returning large objects.

Common interview angle

Interviewers ask:

- What is an rvalue reference?
- What does `std::move` do?
- Is moved-from object destroyed?

Expected answer:

- `std::move` does not move by itself. It casts to an rvalue reference.
- The move constructor or move assignment performs the actual move.
- A moved-from object remains valid but has unspecified state.

4. Real-World Example

Backend server request handling

Imagine a C++ backend server handling user requests.

```
class Connection {
public:
    Connection() {
        std::cout << "Socket opened\n";
    }

    ~Connection() {
        std::cout << "Socket closed\n";
    }
};

void handleRequest() {
    auto buffer = std::make_unique<std::vector<char>>>();
    Connection connection;

    buffer->resize(1024);
    // process request
}
```

What happens:

1. `buffer` owns heap memory safely using `unique_ptr`.
2. `Connection` follows RAII.
3. If an exception occurs, destructors still run.
4. The vector can resize dynamically.
5. Memory and socket resources are cleaned automatically.

In a real backend server, this prevents:

- Memory leaks

- File descriptor leaks
- Socket leaks
- Dangling pointers
- Expensive unnecessary copies

5. Diagrams / Mental Models

Stack vs Heap

Function call stack

local int x		automatic cleanup
local object obj		destructor called when scope ends

Heap memory

object created using new		must be deleted manually
object owned by smart pointer		deleted automatically

Pointer vs Reference

```
int x = 10;
```

Pointer:

p -----> x

Can point somewhere else later.

Can be nullptr.

Reference:

r ===== x

Another name for x.

Cannot be reseated.

Vector resizing

Before push_back:

capacity = 3

[10][20][30]

push_back(40)

New memory allocated:

[10][20][30][40][][]

```
Old memory freed.  
Old pointers and iterators may become invalid.
```

Smart pointer ownership

```
unique_ptr  
  
owner -----> object  
  
Only one owner allowed.  
  
shared_ptr  
  
owner1 ----\  
owner2 -----> object  
owner3 ----/  
  
Object destroyed when reference count becomes 0.
```

6. Common Interview Questions

1. What is the difference between pointer and reference?

Answer

A pointer stores the address of an object. A reference is an alias for an existing object.

Key points interviewer expects

- Pointer can be null.
- Reference should refer to a valid object.
- Pointer can be reassigned.
- Reference cannot be resealed.
- Pointer uses `*` and `->`.
- Reference is often cleaner for function parameters.

Common mistakes

- Saying references are internally always pointers. Implementation is compiler-dependent.
- Saying references can be changed to refer to another object.

2. What is the difference between stack and heap?

Answer

Stack memory is automatically managed for function calls and local variables. Heap memory is dynamically allocated and has flexible lifetime.

Key points interviewer expects

- Stack cleanup is automatic.
- Heap cleanup must be handled manually or by smart pointers.
- Stack allocation is usually faster.
- Heap objects can outlive the function that created them.

Common mistakes

- Saying all local objects are small.
- Saying heap is always bad.
- Forgetting that stack objects can internally manage heap memory.

3. What is the difference between `malloc` and `new`?

Answer

`malloc` allocates raw memory. `new` allocates memory and calls the constructor.

Key points interviewer expects

- `new` is type-safe.
- `malloc` returns `void*`.
- `new` throws exception on failure.
- `malloc` returns `NULL`.
- `new` must be paired with `delete`.
- `malloc` must be paired with `free`.

Common mistakes

- Using `malloc` for C++ objects.
- Pairing `new` with `free`.

4. What is the difference between `delete` and `free`?

Answer

`delete` calls the destructor and releases memory allocated by `new`. `free` releases raw memory allocated by `malloc`.

Key points interviewer expects

- `delete` calls destructor.
- `free` does not call destructor.
- Mixing allocation and deallocation methods is undefined behavior.

Common mistakes

- Saying `free` can safely delete C++ objects.

- Forgetting `delete[]` for arrays allocated with `new[]`.

5. What is RAI?

Answer

RAII is a C++ technique where resources are acquired in constructors and released in destructors.

Key points interviewer expects

- Constructor acquires resource.
- Destructor releases resource.
- Works with scope exit.
- Helps exception safety.
- Used by smart pointers, locks, files, and containers.

Common mistakes

- Explaining RAII only as memory management.
- Forgetting that RAII applies to any resource, not just memory.

6. What is a memory leak?

Answer

A memory leak happens when allocated memory is no longer reachable or is never freed.

Key points interviewer expects

- Usually happens with heap allocation.
- Long-running systems are badly affected.
- Prevent using RAII and smart pointers.
- Detect using tools like Valgrind or sanitizers.

Common mistakes

- Thinking leaks only matter if the program crashes immediately.
- Not mentioning ownership.

7. What is a dangling pointer?

Answer

A dangling pointer points to memory whose lifetime has ended.

Key points interviewer expects

- Can happen after `delete`.
- Can happen when returning address of a local variable.
- Dereferencing it is undefined behavior.

- Smart pointers and lifetime discipline help prevent it.

Common mistakes

- Confusing dangling pointer with null pointer.
- Saying setting one pointer to `nullptr` fixes all aliases.

8. What happens when a vector resizes?

Answer

When a vector needs more capacity, it allocates a larger memory block, moves or copies elements, and destroys the old elements in the old block.

Key points interviewer expects

- Vector stores elements contiguously.
- Reallocation may invalidate iterators, pointers, and references.
- `push_back` is amortized $O(1)$.
- `reserve` can reduce reallocations.

Common mistakes

- Assuming vector never invalidates references.
- Confusing `size()` and `capacity()`.

9. What is the difference between `map` and `unordered_map`?

Answer

`map` stores keys in sorted order using a tree. `unordered_map` stores keys using a hash table without sorted order.

Key points interviewer expects

- `map`: $O(\log n)$, sorted keys.
- `unordered_map`: average $O(1)$, worst $O(n)$.
- `unordered_map` needs hashing.
- `map` supports ordered traversal and range queries.

Common mistakes

- Saying `unordered_map` is always faster.
- Forgetting worst-case hash collisions.

10. What is a virtual function?

Answer

A virtual function allows a derived class function to be called through a base class pointer or reference at runtime.

Key points interviewer expects

- Enables runtime polymorphism.
- Uses dynamic dispatch.
- Usually implemented using vtable and vptr.
- Mark overrides with `override`.

Common mistakes

- Confusing function overloading with overriding.
- Forgetting that dynamic dispatch needs pointer or reference access.

11. Why should a base class destructor be virtual?

Answer

If a derived object is deleted through a base class pointer, the base destructor must be virtual to ensure the derived destructor runs.

Key points interviewer expects

- Required for polymorphic deletion.
- Prevents incomplete destruction.
- Avoids undefined behavior.

Common mistakes

- Thinking virtual destructor is always required for every class.
- Forgetting it in base classes with virtual functions.

12. What is the difference between copy and move?

Answer

Copy duplicates data or resources. Move transfers ownership of resources from one object to another.

Key points interviewer expects

- Copy can be expensive.
- Move is often cheaper.
- `std::move` casts to an rvalue reference.
- Moved-from object remains valid but unspecified.

Common mistakes

- Saying `std::move` itself moves data.

- Using moved-from objects as if their old value is guaranteed.

7. Deep-Dive Questions

1. Why is mixing `new` with `free` undefined behavior?

`new` does two things: allocates memory and constructs the object. `delete` does the reverse: calls the destructor and releases memory. `free` only releases memory and does not call the destructor. Also, the allocator metadata used by `new` and `malloc` may differ. Mixing them breaks the allocation/deallocation contract.

2. How can `shared_ptr` cause a memory leak?

`shared_ptr` uses reference counting. If two objects own each other using `shared_ptr`, their reference counts may never reach zero.

```
struct B;  
  
struct A {  
    std::shared_ptr<B> b;  
};  
  
struct B {  
    std::shared_ptr<A> a;  
};
```

Use `std::weak_ptr` for one side of the relationship to break the cycle.

3. What happens if a vector stores objects and resizing occurs?

When resizing requires reallocation:

1. New memory is allocated.
2. Existing elements are moved if move constructor is available and safe.
3. Otherwise, elements may be copied.
4. Old elements are destroyed.
5. Old memory is released.

This is why efficient move constructors matter.

4. What is object slicing?

Object slicing happens when a derived object is copied into a base object by value. The derived-specific part is lost.

```
class Base {  
public:  
    virtual void show() {}  
};
```

```
class Derived : public Base {
public:
    int extra = 10;
};

Derived d;
Base b = d; // slicing
```

Use base references, pointers, or smart pointers for polymorphism.

5. Why should destructors usually not throw exceptions?

Destructors are often called during stack unwinding after an exception. If a destructor throws while another exception is already active, the program may call `std::terminate`. Destructors should clean up safely and avoid throwing.

8. Comparison Tables

Pointer vs Reference

Feature	Pointer	Reference
Meaning	Stores address	Alias for object
Null allowed	Yes	Normally no
Reassignment	Can point elsewhere	Cannot be reseated
Syntax	<code>*p, p->x</code>	Used like normal variable
Initialization	Can be uninitialized	Must be initialized
Best use	Optional object, dynamic data structures	Function parameters, aliases

Stack vs Heap

Feature	Stack	Heap
Management	Automatic	Manual or smart pointer
Speed	Usually faster	Usually slower
Lifetime	Scope-based	Programmer-controlled
Size	Limited	Larger
Common use	Local variables	Dynamic objects
Risk	Stack overflow	Memory leak, fragmentation

`malloc` vs `new`

Feature	<code>malloc</code>	<code>new</code>
Language	C	C++
Constructor called	No	Yes
Return type	<code>void*</code>	Typed pointer
Failure	Returns <code>NULL</code>	Throws <code>std::bad_alloc</code>
Deallocation	<code>free</code>	<code>delete</code>
Recommended in C++	Rarely	Prefer smart pointers instead

`delete` vs `free`

Feature	<code>delete</code>	<code>free</code>
Used with	<code>new</code>	<code>malloc</code> , <code>calloc</code> , <code>realloc</code>
Destructor called	Yes	No
Type awareness	Yes	No
Array form	<code>delete[]</code>	Same <code>free</code>
Mixing allowed	No	No

`unique_ptr` vs `shared_ptr`

Feature	<code>unique_ptr</code>	<code>shared_ptr</code>
Ownership	Single owner	Multiple owners
Copy allowed	No	Yes
Move allowed	Yes	Yes
Overhead	Low	Higher due to reference count
Destruction	When owner dies	When last owner dies
Best use	Default choice for ownership	Shared lifetime required

`map` vs `unordered_map`

Feature	<code>map</code>	<code>unordered_map</code>
Internal structure	Balanced tree	Hash table
Ordering	Sorted by key	No sorted order
Search	$O(\log n)$	Average $O(1)$, worst $O(n)$
Range queries	Good	Not suitable
Custom requirement	Comparator	Hash function and equality

Feature	map	unordered_map
Best use	Ordered traversal, range queries	Fast lookup

Copy vs Move

Feature	Copy	Move
Meaning	Duplicate resource	Transfer resource
Cost	Can be expensive	Usually cheaper
Source object	Unchanged	Valid but unspecified
Function used	Copy constructor/assignment	Move constructor/assignment
Common trigger	Lvalue	Rvalue or <code>std::move</code>

9. Common Mistakes

- Using `free` for memory allocated by `new`.
- Using `delete` for memory allocated by `malloc`.
- Forgetting `delete[]` for arrays allocated with `new[]`.
- Returning address of a local variable.
- Dereferencing a pointer after `delete`.
- Assuming a pointer becomes `nullptr` automatically after `delete`.
- Thinking references can be resealed.
- Thinking `std::move` always moves data immediately.
- Forgetting virtual destructor in polymorphic base classes.
- Assuming `unordered_map` is always O(1).
- Confusing vector `size()` with `capacity()`.
- Keeping pointers or iterators into a vector after reallocation.
- Overusing `shared_ptr` when `unique_ptr` is enough.
- Creating circular references with `shared_ptr`.

10. Edge Cases / Special Cases

Deleting a null pointer

```
int* p = nullptr;
delete p; // safe
```

Deleting `nullptr` is safe.

Double delete

```
int* p = new int(10);
delete p;
```

```
delete p; // undefined behavior
```

Double deletion is a serious bug.

`new[]` must use `delete[]`

```
int* arr = new int[5];
delete[] arr;
```

Using plain `delete` for arrays is wrong.

Moved-from object

```
std::string a = "hello";
std::string b = std::move(a);
```

`a` is still valid, but its value is unspecified.

`vector::reserve` vs `vector::resize`

```
std::vector<int> v;
v.reserve(100); // capacity becomes at least 100, size remains 0
v.resize(100); // size becomes 100
```

`reserve` changes capacity. `resize` changes size.

Virtual dispatch in constructors and destructors

Virtual calls inside constructors and destructors do not behave like normal polymorphic calls. During base construction, the derived part is not ready yet.

`shared_ptr` from raw pointer twice

```
int* raw = new int(10);
std::shared_ptr<int> a(raw);
std::shared_ptr<int> b(raw); // wrong: two control blocks
```

This can cause double deletion. Prefer `std::make_shared`.

11. How to Explain in Interview

C++ gives direct control over memory and object lifetime. Stack objects are automatically destroyed when scope ends, while heap objects need clear ownership. Raw pointers can represent addresses, but they do not

express ownership clearly. Modern C++ prefers RAI and smart pointers, especially `unique_ptr` for single ownership and `shared_ptr` only when lifetime is truly shared. For polymorphism, virtual functions enable runtime dispatch, and base classes used polymorphically should have virtual destructors. Efficient C++ also depends on understanding vector reallocation, map vs unordered_map tradeoffs, and copy vs move semantics.

12. Quick Revision Notes

Key definitions

- Pointer: variable storing an address.
- Reference: alias for an existing object.
- Stack: automatic storage for local scope.
- Heap: dynamic storage with flexible lifetime.
- RAI: acquire resource in constructor, release in destructor.
- Memory leak: allocated memory is not released.
- Dangling pointer: pointer to invalid memory.
- Virtual function: function resolved at runtime.
- Virtual destructor: destructor allowing correct polymorphic deletion.
- Move: transfer resource instead of copying it.

Important points

- Prefer RAI over manual cleanup.
- Prefer `std::make_unique` and `std::make_shared`.
- Use `unique_ptr` by default for ownership.
- Use `shared_ptr` only for shared lifetime.
- Use `weak_ptr` to break shared pointer cycles.
- `vector` reallocation invalidates old pointers and iterators.
- `map` is ordered; `unordered_map` is hash-based.
- `std::move` is a cast, not the move operation itself.

Common comparisons

- Pointer vs reference
- Stack vs heap
- `malloc` vs `new`
- `delete` vs `free`
- `unique_ptr` vs `shared_ptr`
- `map` vs `unordered_map`
- Copy vs move

Must-remember facts

- Match `new` with `delete`.
- Match `new[]` with `delete[]`.
- Match `malloc` with `free`.
- Deleting through a base pointer needs a virtual destructor.
- `unordered_map` has average O(1), not guaranteed O(1).

- A moved-from object is valid but unspecified.

Interview traps

- `free` does not call destructors.
- `malloc` does not call constructors.
- `delete` does not automatically set pointer to `nullptr`.
- `shared_ptr` can leak due to cycles.
- References cannot be reseated.
- Vector capacity and size are different.

13. Practice Tasks

Task 1: Pointer and reference

Write two functions:

```
void updateByPointer(int* p);  
void updateByReference(int& r);
```

Call both and observe how the original value changes.

Task 2: Memory leak detection

Write a program that intentionally leaks memory using `new`. Then fix it using `delete`, and then rewrite it using `std::unique_ptr`.

Task 3: RAll class

Create a class `Logger` that prints:

- `Logger started` in constructor
- `Logger stopped` in destructor

Create it inside a function and observe when the destructor runs.

Task 4: Vector resizing

Write code that prints vector `size()` and `capacity()` after each `push_back`.

```
std::vector<int> v;  
for (int i = 0; i < 20; i++) {  
    v.push_back(i);  
    std::cout << v.size() << " " << v.capacity() << "\n";  
}
```

Task 5: Iterator invalidation

Store a pointer to `v[0]`, keep pushing elements, and observe whether the address changes after reallocation.

Task 6: `map` vs `unordered_map`

Insert the same key-value pairs into both containers and print the keys. Observe that `map` prints sorted keys while `unordered_map` does not.

Task 7: Virtual destructor experiment

Create a base class and derived class. Delete derived object through base pointer once with a non-virtual destructor and once with a virtual destructor. Observe the destructor calls.

Task 8: Copy vs move

Create a class with copy constructor and move constructor that print messages. Push objects into a vector and observe when copy or move happens.

Task 9: `shared_ptr` cycle

Create two classes that hold `shared_ptr` to each other. Observe that destructors are not called. Then replace one side with `weak_ptr`.

Task 10: Placement-style explanation

Practice explaining these in under 60 seconds:

- Pointer vs reference
- Stack vs heap
- RAI
- `unique_ptr` vs `shared_ptr`
- Copy vs move

14. Final Cheat Sheet

Core definition

C++ memory management is about controlling where objects live, who owns them, and when they are destroyed.

Why it matters

It prevents leaks, crashes, dangling pointers, double deletes, resource leaks, and unnecessary copying.

Most asked questions

- Pointer vs reference
- Stack vs heap
- `malloc` vs `new`
- `delete` vs `free`
- What is RAI?
- `unique_ptr` vs `shared_ptr`

- What is a memory leak?
- What is a dangling pointer?
- What happens during vector resizing?
- `map` vs `unordered_map`
- Why virtual destructor?
- Copy vs move

Common comparisons

Topic	One-line difference
Pointer vs Reference	Pointer stores address; reference is alias
Stack vs Heap	Stack is automatic; heap is dynamic
<code>malloc</code> vs <code>new</code>	<code>malloc</code> allocates bytes; <code>new</code> constructs objects
<code>delete</code> vs <code>free</code>	<code>delete</code> calls destructor; <code>free</code> does not
<code>unique_ptr</code> vs <code>shared_ptr</code>	Single ownership vs shared ownership
<code>map</code> vs <code>unordered_map</code>	Sorted tree vs hash table
Copy vs Move	Duplicate resource vs transfer resource

One-line interview answer

In modern C++, safe memory management is mainly about RAI: use stack objects and smart pointers to express ownership clearly, avoid raw owning pointers, understand container invalidation, and use copy or move semantics correctly for performance and safety.